

Model testare - Implementarea Concurenței în Limbajele de Programare

Opțional 2024-2025

Materiale permise: materialele de curs și cele de laborator, inclusiv fișiere cod cu soluții ale problemelor rezolvate în curs și laborator; în plus, fiecare student poate avea o foaie (2 pagini) în format fizic (scris de mână/printat) cu elemente de limbaj (Haskell, Java).

În timpul examenului NU AVEȚI VOIE SĂ AVEȚI BROWSERE DESCHISE, NU PUTEȚI CONSULTA MATERIALE ONLINE.

Fiecare student va avea un folder pe care îl va denumi "nume-student-ICLP-2025" în care va avea aceste materiale și în care va rezolva exercițiile. În acest folder, pe parcursul examenului NU pot să apară alte fișiere în afara celor menționate.

La finalul examenului veți face submiterea subiectelor rezolvate NUMAI în prezența unui supraveghetor. Vă vom anunța în timpul examenului modul de submitere.

1 Partea 0x00 - Java (maxim 3 puncte)

Se pot alege și limbajele Python, respectiv Go pentru rezolvarea acestui subiect.

1.1 Testare 2024

Sa se scrie un program care citește de la **STDIN** un *path* absolut, doua numere întregi $2 < N < 10$ și $3 < M < 8$ și implementează următorul scenariu: prin intermediul a N *thread*-uri cu rol dedicat, vor fi citite toate fișierele text continute în *path*-ul specificat, inclusiv în subfoldere (*vezi codul auxiliar de mai jos*), iar conținutul lor va fi scris de alte M *thread*-uri la **STDOUT**, pastrand doar caracterele alfanumerice. Implementați și un monitor care să retina toate *log*-urile, cu informații de forma: **Thread Id {threadId} is currently working on file {filePath}** sau **Thread Id {threadId} is currently writing at standard output**. Implementați un mecanism astfel încât scrierea la **STDOUT** să fie efectuată după prelucrarea a cel puțin 2 fișiere text. Justificați, într-un comentariu, modul în care ați decis să implementați acest mecanism.

Cod auxiliar.

```
class FileWalker {
    public ArrayList<String> walk(String path) {
        ArrayList<String> files = new ArrayList<>();

        File root = new File(path);
        File[] list = root.listFiles();

        if (list == null) {
            return files;
        }

        for (File f : list) {
            if (f.isDirectory()) {
                files.addAll(walk(f.getAbsolutePath()));
            }
            else if (f.getName().endsWith(".txt")) {
                files.add(f.getAbsolutePath());
            }
        }

        return files;
    }
}
```

1.2 Testare 2023

Folosind **ExecutorService**, implementați un **array** de acțiuni care să se execute asincron. În executarea propriu-zisă a acestor acțiuni, logați, la **STDOUT**, și indexul acțiunii care s-a executat, nu doar rezultatul.

- Particularizați scenariul pentru calculul în paralel a funcției **fibonacci**. Cele N poziții din **array**-ul **v** vor executa **fibonacci(v[i])**, $0 \leq i < N$.
- Particularizați scenariul într-un context **Reader-Writer**, astfel:

- Fiecare acțiune execută un **Reader** și poate să citească linii dintr-o bază de date, între două limite, N și M . Se garantează $N < M$.
- După ce au citit din baza de date, peste liniile selectate se aplică o funcție **computeResult**. Cu ajutorul **thread**-urilor **Writer** se actualizează sursa de date, etichetând fiecare linie cu scorul **batch**-ului. În același timp, toate rezultatele se trimit sub forma (**index**, **result**) unui alt **thread**, cu rolul de monitor, care reține mereu doar perechea corespunzătoare celui mai mare scor obținut.
- După ce toate acțiunile s-au executat, **thread**-ul monitor afișează la **STDOUT** indexul acțiunii cu cel mai bun scor.

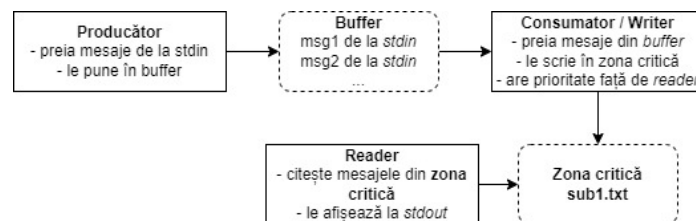
Observație: Pentru implementare, citirea/scrierea/actualizarea în baza de date vor fi acțiuni care doar vor dura un anumit timp (**random**) și vor respecta o semnătură intuitivă. Trebuie doar să implementați arhitectura acestui scenariu: modul de funcționare pentru **readeri** și **writeri**, respectiv sistemul final de vot, care doar calculează maximumul dintre rezultatele funcției pe **batch**-uri. Funcția **computeResult** poate fi implementată să returneze un **random**.

1.3 Testare 2022

Se citesc mesaje de la *standard input*. Să se implementeze un scenariu concurent care efectuează următoarele:

1. mesajele vor fi citite într-un model *Producer-Consumer*;
2. *consumatorul* din acest model va fi un *Writer* într-un model *Reader-Writer*. În cadrul acestuia, *thread*-urile *Writer* vor avea prioritate față de *thread*-urile *Reader*. Mesajele citite vor fi scrise de *writeri* într-un fișier *sub1.txt* (fiecare *writer* va adăuga mesajul său în această zonă critică), iar *readerii* vor afișa, la *standard output*, conținutul respectivului fișier.

Important! *Thread*-urile *Reader* nu pot porni până ce nu avem garanția că a fost cel puțin mesaj scris în zona critică.



2 Partea 0x01 - Haskell (maxim 3 puncte)

2.1 Testare 2024

O companie cu 50 de angajați și 5 secretare are un spațiu care conține o zonă de conferințe și o zonă de printare. Zona de printare este folosită de una din cele 5 secretare ale companiei, dar secretarele nu pot folosi zona simultan. Zona de conferințe are 10 locuri, accesul participantilor fiind permis numai când grupul de angajați este complet. Angajații încearcă să participe la o conferințe de mai multe ori pe parcursul unei zile. Fiecare secretară trebuie să printeze de mai multe ori pe parcursul unei zile. Accesul la cele două zone este gestionat de un administrator, care nu lasa angajații și secretarele să desfășoare activități simultan. Scrieți un program în Haskell care să implementeze accesul la spațiul descris anterior.

2.2 Testare 2023

Într-o intersecție aglomerată, este montat un semafor inteligent care să ajute la eficientizarea traficului. Semaforul are montată o cameră prin care poate recunoaște mașini, pietoni și animale. Modul în care semaforul permite trecerea prin intersecție este următorul:

- va permite trecerea grupurilor de pietoni, când există 5 pietoni în așteptare să traverseze (semaforul va deveni roșu pentru mașini);
- va permite trecerea mașinilor, când există 10 mașini în așteptare să treacă prin intersecție (semaforul va deveni verde pentru mașini);
- va opri trecerea mașinilor de fiecare dată când va recunoaște un animal care vrea să treacă strada (semaforul va deveni roșu pentru mașini).

Semaforul va funcționa cu următoarea restricție: animalele și pietonii au prioritate în fața mașinilor. În scenariul prezentat, grupurile de animale, pietoni și mașini vor veni la infinit.

2.3 Testare 2022

Definim un canal de comunicare printr-o variabilă `MVar` care conține o listă de întregi. Mesajele care se pot scrie / citi folosind acest canal vor fi numere întregi.

```
import Control.Concurrent
data MList = ML (MVar [Int])
```

Scrieți un program concurent care lansează două *thread*-uri, unul care citește, respectiv unul care scrie, astfel:

- *thread*-ul care scrie, va scrie începând din stânga canalului întregii introduși la *standard input*;
- *thread*-ul care citește, va citi din dreapta canalului, va afișa la *standard output* dublul numărului citit, și va elimina informația prelucrată.

Programul își va înceta executarea în momentul în care se va citi întregul 0. Nu se vor face verificări suplimentare, presupunem că utilizatorul introduce date de tipul așteptat (`Int`). Reamintim că aceste date sunt citite de la *standard input* ca șiruri de caractere.

3 Partea 0x02 - Erlang (maxim 3 puncte)

Modelati un scenariu in care avem o cladire cu doua lifturi si mai multi oameni care interactioneaza cu cele doua lifturi. Cineva poate sa solicite un lift pentru a merge la un anumit etaj, iar lifturile trebuie sa raspunda cererilor. Consideram, pentru lifturi, starile `stationary(Floor)` (liftul stationeaza la un anumit etaj), `moving(Direction, Floor)` (liftul se deplaseaza spre un anumit etaj, iar `Direction` poate fi `up` ori `down`). Pentru fiecare persoana, avem starile `waiting(Floor, Destination)` (asteapta la un anumit etaj, pentru a ajunge la un altul), respectiv `in_lift(Lift, Destination)` (in ce lift se afla, si la ce etaj vrea sa ajunga) Fiecare om trimite cererea catre ambele lifturi, si va fi prioritar liftul care e cel mai apropiat. Lifturile trebuie sa termine cate un transport pentru a putea prelua noi cereri. Lifturile au, initial, o capacitate maxima.